



Introduction au langage de programmation Python

Séance 1 - Les bases de la programmation



Programme du cours

Séance	Date	Sujet
1	29 septembre 2026	Les bases de la programmation
2	6 octobre 2026	Créer ses propres outils : fonctions et modules
3	13 octobre 2026	Programmation orientée objet 1 : créer ses propres objets
4	20 octobre 2026	Programmation orientée objet 2 : héritage et décorateurs
5	27 octobre 2026	Lire et enregistrer des données : JSON
6	3 novembre 2026	Du tableur à Python : CSV
7	10 novembre 2026	Explorer les collections numérisées : IIIF



Objectifs du cours : apprendre une nouvelle langue

Objectif : savoir lire du code et le comprendre, avant de savoir l'écrire. On procède comme pour une nouvelle langue, la compréhension précède la production.

```
for ms in corpus:  
    print(ms["cote"])
```

=> « Pour chaque manuscrit du corpus, affiche sa cote. »

Langue naturelle	Python
L'alphabet : les signes	Les symboles et la syntaxe : <code>=</code> , <code>" "</code> , <code>[]</code> , <code>{}</code> , <code>:</code> , l'indentation
Les noms : ce dont on parle	Les données : variables, nombres, textes, listes, dictionnaires
Les verbes : ce que l'on fait	Les fonctions et méthodes : <code>print()</code> , <code>len()</code> , <code>append()</code>
La grammaire : assembler des phrases	Les structures de contrôle : conditions et boucles



Plan de la séance 1

- 1) Présentation du langage Python
- 2) Installation de Python et de VS Code
- 3) Les bases de la programmation
 - a) Variables et types de données
 - i) Types de base : entiers, flottants, chaînes (et f-strings), booléens
 - ii) Listes, tuples, dictionnaires, sets
 - b) Manipuler les données
 - i) Concaténation, index, tranches
 - ii) Fonctions et méthodes utiles : `type()`, `len()`, `split()`, `sort()`, `append()`, `keys()`, `values()`
 - c) Structures de contrôle
 - i) Opérateurs de comparaison et opérateurs logiques
 - ii) Conditions : `if`, `elif`, `else`
 - iii) Boucles bornées : `for` (avec `range()`)
 - iv) Boucles non bornées : `while` (avec `break` et `continue`)



Qu'est ce que Python ?

Création et Origine

- Créé en 1991 par [Guido van Rossum](#)
- Nom inspiré par la troupe comique britannique "Monty Python's Flying Circus"
- Développé comme un successeur du langage [ABC](#), avec l'objectif de créer un langage simple, lisible et puissant pour le prototypage et le développement rapide
- Philosophie : Mettre l'accent sur la clarté du code, la simplicité d'utilisation et une communauté ouverte



« ... In December 1989, I was looking for a **“hobby” programming project that would keep me occupied during the week around Christmas**. My office ... would be closed, but I had a home computer, and not much else on my hands. I decided to write an interpreter for the new scripting language I had been thinking about lately: a descendant of ABC that would appeal to Unix/C hackers. I chose Python as a working title for the project, being in a slightly irreverent mood (and a big fan of Monty Python's Flying Circus). »



Qu'est ce que Python ? Un langage de programmation de Haut Niveau

Langage de Haut Niveau

Intérêt

- **Facilité d'écriture** : Plus simple de coder qu'avec un langage proche de la machine.
- **Rapidité de développement** : Programmes plus rapides à écrire et à déboguer.
- **Portabilité accrue** : Les programmes sont plus facilement transférables d'une machine à une autre grâce à un niveau d'abstraction élevé des particularités des processeurs.

Inconvénients

- **Moins performants** : Coût plus élevé en termes de ressources (temps d'exécution, mémoire).
- **Moins de contrôle direct** sur le matériel.

```
print("Hello World")
```

Langages de Bas Niveau (ex. C)

Intérêt

- **Proches du langage machine** : instructions proches du code binaire exécuté par l'ordinateur
- **Contrôle précis du matériel** : gestion directe de la mémoire, des registres et du processeur
- **Très performants** : utilisés pour les systèmes d'exploitation, les pilotes ou les logiciels embarqués

Inconvénients

- **Mais complexes** : nécessitent une connaissance approfondie de l'architecture matérielle et une gestion manuelle des **ressources (mémoire, pointeurs, etc.)**

```
#include <stdio.h>

int main() {
    printf("Hello World\n");
    return 0;
}
```



Le zen de python

Beautiful is better than ugly.

Explicit is better than implicit.

Simple is better than complex.

Complex is better than complicated.

Flat is better than nested.

Sparse is better than dense.

Readability counts.

Special cases aren't special enough to break the rules.

Although practicality beats purity.

Errors should never pass silently.

Unless explicitly silenced.

In the face of ambiguity, refuse the temptation to guess.

There should be one-- and preferably only one --obvious way to do it.

Although that way may not be obvious at first unless you're Dutch.

Now is better than never.

Although never is often better than *right* now.

If the implementation is hard to explain, it's a bad idea.

If the implementation is easy to explain, it may be a good idea.

Namespaces are one honking great idea -- let's do more of those!





Installation de Python

Linux (Ubuntu)	macOS	Windows
1. Ouvrir le Terminal.	1. Aller sur python.org/downloads .	1. Aller sur python.org/downloads .
2. Mettre à jour les paquets : <code>sudo apt update</code>	2. Télécharger la dernière version stable pour macOS.	2. Télécharger la dernière version stable pour Windows.
3. Installer Python et pip : <code>sudo apt install python3 python3-pip</code>	3. Ouvrir le fichier <code>.pkg</code> et suivre l'installateur.	3. Lancer l'installateur. Si l'option apparaît, cocher « Add python.exe to PATH ».
4. Vérifier : <code>python3 --version</code>	4. Vérifier dans le Terminal : <code>python3 --version</code>	4. Vérifier dans PowerShell : <code>python --version</code>



Installation de Visual Studio Code (VSCode)

Linux (Ubuntu/Debian)

- Télécharger **.deb (64-bit)** depuis code.visualstudio.com
- Installer : `sudo dpkg -i code_*.deb` puis
`sudo apt-get install -f`

macOS

- Télécharger le fichier **.zip** depuis code.visualstudio.com
- Glisser **VS Code.app** dans **Applications**

Windows

- Télécharger l'installateur (User Installer) sur code.visualstudio.com
- Lancer l'installateur et suivre les étapes

```
1 import torch
2 import whisper
3 import gradio as gr
4 import tempfile
5 import os
6 import json
7
8 def load_whisper_model(model_file=None, model_name=None):
9
10     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
11
12     # 1st case : if a personal model is loaded (.pt)
13     if model_file is not None:
14         try:
15             model_path = model_file.name if hasattr(model_file, 'name') else model_file
16             whisper_model = whisper.load_model(model_path, device=device)
17             print(f'Model loaded from {model_path}')
18             return whisper_model
19
20     except Exception as e:
21         print(f'An error occurred while loading the model: {e}')
22         raise
```



Variables et types de données: basiques

Définition

Les variables sont des conteneurs qui **stockent des données**.
On **assigne** une valeur à un nom avec le signe `=`, puis on peut **réutiliser** ce nom dans la suite du code pour manipuler la valeur ou faire des calculs.

Dans notre analogie, ce sont les **noms** de la langue.

```
bibliotheque = "Aberdeen, University Library"  
cote = "MS 24"  
date = 1200
```

Règles de nommage des variables

- Commencer par une lettre ou un underscore (`_`).
- Utiliser uniquement des **lettres, chiffres, et underscore** (`nom_auteur`, `ms_1`).
- Ne mettre ni **espaces** ni **caractère spécial** (`$`, `@`, etc.), et éviter les accents.
- Ne pas utiliser de **mot réservé**.

False	None	True	and	as	assert	async
await	break	class	continue	def	del	elif
else	except	finally	for	from	global	if
import	in	is	lambda	nonlocal	not	or
pass	raise	return	try	while	with	yield



Variables et types de données : basiques

Types de données de base

Type	Représente	Exemple
<code>str</code> (chaîne de caractères)	du texte	<code>cote = "MS 24"</code>
<code>int</code> (entier)	un nombre sans décimale	<code>date = 1200</code>
<code>float</code> (flottant)	un nombre à décimales	<code>hauteur_cm = 30.5</code>
<code>bool</code> (booléen)	une valeur de vérité : <code>True</code> ou <code>False</code>	<code>numerise = True</code>

Sensibilité à la casse

Les variables en Python sont **sensibles à la casse**, ce qui signifie que `cote`, `Cote` et `COTE` sont **trois variables différentes**.

Les f-strings permettent d'insérer des variables dans une chaîne. On place un `f` avant les guillemets et les variables entre accolades `{}` :

```
print(f"{bibliotheque}, {cote} (vers {date})")
```

```
# Aberdeen, University Library, MS 24 (vers 1200)
```



Listes

Définition : Séquence ordonnées modifiables

Création de liste :

Une liste est toujours contenue dans des crochets `[]` et chaque élément est séparé par une virgule.

Accès aux éléments par index

Les éléments d'une liste possède tous un index qui permet de les extraire en fonction de leur place dans la liste

Méthodes utiles

- 1) `append()` pour ajouter un élément
- 2) `sort()` pour trier la liste : **Attention ne fonctionne que si les éléments sont comparables entre eux, (ex : uniquement des chaînes ou des nombres).**

Créations de liste

```
creatures = ["lion", "panthère", "licorne",  
"dragon"]
```

Requête par index

Index:	0	1	2	3
creatures:	"lion"	"panthère"	"licorne"	"dragon"

```
creatures[0] # "lion"  
creatures[-1] # "dragon" (dernier élément)
```

Ajouter des éléments dans une liste

```
creatures.append("phénix")
```

Trier les éléments de la liste

```
creatures.sort() # ['dragon', 'licorne', 'lion', 'panthère', 'phénix']
```



Tuples : définition et création

Définition : un tuple est une séquence **ordonnée** et **immuable**, c'est-à-dire qu'on ne peut plus la modifier après sa création.

Création de tuples : les éléments sont placés entre **parenthèses ()** et séparés par des virgules.

Accès aux éléments par index

Les éléments d'un tuple, comme pour les listes possèdent tous un index.

Création de tuples

```
datation = (1170, 1200, 1210)
```

Index

```
datation[0]      # 1170
```



Tuples : utilisation

Immuabilité :

- Impossible de modifier, d'ajouter ou de supprimer des éléments après la création.
- Toute tentative de modification provoquera une **erreur**

Utilités des tuples :

- **Stockage de données constantes**, qui ne doivent pas changer : une fourchette de datation, des dimensions.
- **Clés de dictionnaire**, si leurs éléments sont eux-mêmes immuables.
- **Le déballage (unpacking)** : assigner chaque élément du tuple à une variable distincte.

Immuabilité

```
creatures[0] = "phoenix"  
# Erreur: impossible de modifier un  
tuple
```

Déballage

```
datation = (1200, 1210)  
debut, fin = datation  
print(f"Manuscrit daté entre {debut}  
et {fin}")  
# Manuscrit daté entre 1200 et 1210
```



Dictionnaires : définition et création

Définition : un dictionnaire associe des **clés uniques** à des **valeurs**. Chaque clé permet de retrouver rapidement la valeur associée. On l'utilise lorsque les données doivent être consultées par un identifiant, comme les champs d'une notice.

Création : un dictionnaire s'écrit entre accolades `{ }`. Chaque paire clé-valeur est séparée par deux points `:`, et les paires sont séparées par des virgules `,`.

Accès aux valeurs : Les valeurs d'un dictionnaire sont accédées par leurs **clés** uniques.

Ajouter une nouvelle paire clé-valeur : on assigne une valeur à une clé qui n'existe pas encore. Pour ajouter plusieurs paires d'un coup, il existe aussi la méthode `update()`

Création d'un dictionnaire

```
manuscrit = {  
    "cote": "MS 24",  
    "bibliotheque": "Aberdeen, University Library",  
    "date": 1200,  
    "numerise": True }
```

Accès par clé

Clé		"cote"		"bibliotheque"		"date"		"numerise"
Valeur		"MS 24"		"Aberdeen, University Library"		1200		True

Ajouter une nouvelle paire clé-valeur

```
manuscrit["langue"] = "latin"  
manuscrit.update({"langue": "latin", "origine": "Angleterre"})
```

Si la clé existe déjà, la même syntaxe remplace sa valeur :

```
manuscrit["date"] = 1210
```



Dictionnaires : méthodes utiles

Les dictionnaires sont des structures de données très utilisées en Python, car ils permettent d'associer des **clés** à des **valeurs**.

Pour les manipuler facilement, il existe des méthodes pratiques qui offrent un accès direct :

- aux **clés** du dictionnaire
 - `keys()` : Retourne une **liste de toutes les clés** du dictionnaire.
- aux **valeurs** qu'elles contiennent
 - `values()` : Retourne une **liste de toutes les valeurs** du dictionnaire.
- ou encore aux **paires clé-valeur** sous forme de tuples.
 - `items()` : Retourne une **liste de tuples** représentant chaque **paire clé-valeur**.

Méthodes

```
manuscrit.keys()  
# dict_keys(['cote', 'bibliotheque',  
'date', 'numerise', 'langue'])
```

```
manuscrit.values()  
# dict_values(['MS 24', 'Aberdeen,  
University Library', 1200, True,  
'latin'])
```

```
manuscrit.items()  
# dict_items([('cote', 'MS 24'),  
( 'bibliotheque', 'Aberdeen, University  
Library'), ...])
```



Sets (ensembles) : définition et création

Définition : un set est une collection **non ordonnée** d'éléments **uniques**. Les éléments eux-mêmes doivent être immuables. On l'utilise pour stocker plusieurs éléments sans doublon.

Création : Un set s'écrit entre **accolades** `{}` ou la fonction `set()`.

Pour un set vide, on utilise `set()`, car `{}` crée un dictionnaire vide.

Création d'un set

```
creatures = {"lion", "panthère", "licorne", "dragon"}
```

Les doublons disparaissent automatiquement :

```
mentions = ["lion", "dragon", "lion", "licorne", "dragon"]  
set(mentions)  # {'lion', 'dragon', 'licorne'}
```

Création d'un set vide

```
set_vide = set()
```



Sets (Ensembles) : caractéristique et méthodes

Caractéristiques

- **Non ordonnés** : pas d'index, donc pas d'accès par position. L'ordre d'affichage peut varier.
- **Éléments uniques** : un même élément ne peut pas apparaître deux fois.
- **Muabilité** : le set lui-même est modifiable (ajout, suppression), mais ses éléments doivent être immuables (chaînes, nombres, tuples).

Méthodes utiles

- `add(element)` ajoute un élément.
- `remove(element)` supprime un élément, et provoque une erreur s'il n'existe pas.
- `discard(element)` supprime un élément, sans erreur s'il n'existe pas.
- `clear()` vide le set.

```
creatures.add("phénix") # ajoute "phénix"
```

```
creatures.remove("licorne") # supprime  
"licorne"
```

```
creatures.discard("sirène") # pas  
d'erreur, même si "sirène" est absente
```

```
creatures.clear() # vide le set
```



Manipuler les données

La manipulation de variables consiste à créer, modifier, combiner et utiliser des variables pour stocker et traiter des données tout au long du programme : calculs, opérations logiques, manipulations de chaînes.

- **La concaténation** combine plusieurs chaînes en une seule avec l'opérateur `+`.
- **L'indexation** accède à un caractère précis d'une chaîne grâce à son index.
- **Les tranches (slicing)** extraient une sous-chaîne avec un index de début et un index de fin (exclu).

Concaténation

```
titre = "III, 14." + " " + "De la  
colombe."  
# "III, 14. De la colombe."
```

Indexation

```
titre[0]          # "I"
```

Slicing

```
titre[0:3]        # "III"  
titre[-8:-1]      # "colombe"
```



Manipuler les données : fonctions utiles

Certaines fonctions intégrées (*built-in functions*) reviennent très souvent. Elles permettent :

- de connaître le type d'une variable avec `type(variable)` ;
- de mesurer la taille d'une séquence avec `len(sequence)` ;
- de transformer une chaîne en liste avec `split(sep)`.

Notes : une **fonction** s'applique à une donnée, `len(titre)` ; une **méthode** appartient à un type de donnée et s'écrit après un point, `titre.split()`. On y reviendra en séance 2.

Afficher le type

```
type(titre)           # <class 'str'>
```

Mesurer

```
len(titre)           # 23
```

Transformer

```
titre.split(" ")     # ['III,', '14.',  
                      'De', 'la', 'colombe.']
```



Opérateurs de comparaison

Les opérateurs de comparaison comparent deux valeurs et renvoient une valeur de vérité : **True** ou **False**.

Opérateur	Signification	Exemple
<code>==</code>	égal à	<code>manuscrit["date"] == 1200 # True</code>
<code>!=</code>	différent de	<code>manuscrit["langue"] != "français" # True</code>
<code>></code>	strictement supérieur à	<code>manuscrit["date"] > 1200 # False</code>
<code><</code>	strictement inférieur à	<code>manuscrit["date"] <= 1200 # False</code>
<code>>=</code>	supérieur ou égal à	<code>manuscrit["date"] > 1200 # True</code>
<code><=</code>	inférieur ou égal à	<code>manuscrit["date"] <= 1200 # True</code>



Opérateurs logiques

Les opérateurs logiques combinent ou inversent des conditions.

On les utilise dans les structures de contrôle (if, while).

- **ET (and)** : vrai si toutes les conditions sont vraies.
- **OU (or)** : vrai si au moins une condition est vraie.
- **NON (not)** : inverse la valeur de vérité.

Exemples

```
manuscrit["date"] <= 1200 and manuscrit["numerise"]      # True
manuscrit["langue"] == "latin" or manuscrit["date"] > 1300 # True
not manuscrit["numerise"]                                  # False
```



Structures de contrôle : les conditions

Les structures conditionnelles exécutent des sections de code différentes selon que des conditions sont vraies ou fausses.

Elles utilisent les mots-clés `if`, `elif` et `else`.

- `if` teste une première condition.
- `elif` teste une condition supplémentaire, si les précédentes sont fausses.
- `else` s'exécute si aucune des conditions précédentes n'est vraie.

Gestion de plusieurs cas avec `elif` et `else` :

Les mots-clés `elif` et `else` permettent de gérer **plusieurs conditions** successives.

- Si une seule condition doit être vraie → `elif` est le bon choix
- Si plusieurs conditions peuvent être vraies en même temps → utilisez plusieurs `if`

```
if condition_1:
    # exécuté si condition_1 est vraie
elif condition_2:
    # exécuté si condition_1 est
    # fausse et condition_2 vraie
else:
    # exécuté si aucune condition
    # n'est vraie
```



Syntaxe d'une condition : l'ordre compte

```
date = 1150
```

```
if date <= 1100:
    print("XIe siècle ou
avant")
elif date <= 1200:
    print("XIIe siècle")
elif date <= 1300:
    print("XIIIe siècle")
else:
    print("Après 1300")
# XIIe siècle
```

Et non :

```
if date <= 1300:
    print("XIIIe siècle")
elif date <= 1200:
    print("XIIe siècle")
# XIIIe siècle : faux !
```

La première condition vraie l'emporte, et les suivantes ne sont pas testées.

Structures de contrôle : l'indentation

L'indentation est essentielle en Python : elle définit les blocs de code associés à une condition.

- Chaque bloc placé sous une condition est décalé vers la droite, généralement de 4 espaces.
- Une mauvaise indentation provoque une erreur.
- VS Code indente automatiquement, mais il faut rester attentif quand plusieurs structures sont imbriquées.

Conseils :

- Toujours s'assurer de **l'indentation correcte** pour chaque bloc de code conditionnel.
- Penser à utiliser le mot-clé **else** pour couvrir les cas non prévus dans les conditions précédentes.

```
if manuscrit["numerise"]:  
    print("Consultable en ligne")  
else:  
    print("Consultation sur place")
```



Boucles bornées **for** : définition et syntaxe

La boucle **for** répète un ensemble d'instructions pour chaque élément d'une séquence (liste, tuple, chaîne de caractères, etc.). On l'utilise pour **parcourir** une collection ou pour **répéter des instructions** un nombre défini de fois.

Syntaxe de la Boucle **for** :

- **element** : la variable qui prend tour à tour la valeur de chaque élément.
- **sequence** : la collection parcourue.

Exemple de boucle **for**

```
for element in sequence:  
    # instructions exécutées pour  
    chaque élément
```

```
creatures = ["dragon", "licorne",  
             "lion", "panthère", "phénix"]
```

```
for creature in creatures:  
    print(creature)
```



Boucles bornées : `range()`

Utilisation avec `range()` :

La fonction `range()` génère une **suite de nombres**, utilisable dans une boucle `for`.

- **Syntaxe** : `range(start, stop, step)`
 - **start** : valeur de départ, incluse (0 par défaut).
 - **stop** : valeur de fin, exclue.
 - **step** : pas entre les valeurs (1 par défaut).

Conseils :

- Les boucles avec `range()` sont pratiques pour les **répétitions** et les **comptages**.

Exemples avec `range`

```
for i in range(5):  
    print(i)
```

```
for i in range(2, 10, 2):  
    print(i)
```

```
for f in range(1, 6):  
    print(f"f. {f}")  
# f. 1, f. 2, f. 3, f. 4, f. 5
```

```
for f in range(1, 10, 2):  
    print(f"f. {f}")  
# f. 1, f. 3, f. 5, f. 7, f. 9
```



Instructions **break** et **continue**

Les instructions **break** et **continue** sont utilisées pour **contrôler le flux** d'exécution d'une boucle **for** ou **while**. Elles permettent de modifier le comportement standard d'une boucle en fonction de certaines conditions.

Instruction **break** :

L'instruction **break** permet d'**interrompre immédiatement** la boucle dans laquelle elle se trouve, peu importe si toutes les itérations ne sont pas terminées.

- Utilisation : **Sortir** d'une boucle lorsqu'une certaine condition est remplie.

Instruction **continue** :

L'instruction **continue** permet de **passer immédiatement** à l'**itération suivante** de la boucle, en **ignorant** les instructions restantes pour l'itération courante.

- Utilisation : **Sauter** une partie du code dans une boucle lorsqu'une condition est remplie.



break et continue : exemples

`break` et `continue` contrôlent le déroulement d'une boucle `for` ou `while`.

- **break** interrompt immédiatement la boucle, même si toutes les itérations ne sont pas terminées. On l'utilise pour sortir de la boucle dès qu'une condition est remplie, par exemple pour trouver un élément.
- **continue** passe immédiatement à l'itération suivante, en ignorant les instructions restantes. On l'utilise pour ignorer certaines valeurs.

Conseils : utiliser `break` avec précaution pour ne pas interrompre une boucle trop tôt ; `continue` permet souvent de simplifier le code.

Utilisation de `break`

```
for creature in creatures:
    if creature == "licorne":
        print("Licorne trouvée, arrêt de
la recherche.")
        break    # sort de la boucle dès
que la licorne est trouvée
python
```

Utilisation de `continue`

```
for mot in titre.split(" "):
    if len(mot) <= 2:
        continue    # ignore les mots
courts ("De", "la")
    print(mot)
# III, 14.  colombe.
```



Boucles non bornées : `while`

La boucle `while` répète un ensemble d'instructions tant qu'une condition est vraie. Elle convient quand le nombre d'itérations n'est pas connu à l'avance.

Syntaxe

- **Condition** : une expression évaluée à `True` ou `False`. La boucle continue tant qu'elle est vraie.
- **Instructions** : le bloc exécuté à chaque itération.

Syntaxe de la boucle

```
mots = titre.split(" ")
i = 0
while mots[i] != "colombe.":
    i += 1    # passe au mot suivant
print(f"« colombe » est à l'index {i}")
# « colombe » est à l'index 4
```



Boucles non bornées : boucles infinies

Une boucle **while** sans condition de sortie appropriée peut devenir **infinie**, car la condition reste **toujours vraie**. Il est important d'avoir une **instruction de sortie** comme **break** ou de modifier la condition pour éviter un blocage.

Conseils

- Évitez les boucles infinies en s'assurant que la condition finisse par être fausse ou en ajoutant un **break**.
- N'utilisez **while** lorsque le nombre d'itérations n'est **pas connu** à l'avance.
- Les boucles **while** sont parfaites pour **attendre une condition** spécifique avant de continuer.

Boucles infinies

```
while True:
    print("Ceci est une boucle infinie") # S'exécute en continue
```



Notions clés à retenir

Types de données

- Types de base : entiers (`int`), flottants (`float`), chaînes de caractères (`str`) et booléens (`bool`).
- Structures de données : listes, tuples, dictionnaires et sets, pour organiser et gérer les données.
- Savoir déclarer, assigner et réutiliser une variable.

Manipulation des données

- Concaténation, index et tranches sur les chaînes.
- Méthodes propres à chaque structure : `append()`, `sort()`, `keys()`, `add()`, etc.
- Fonctions utiles : `type()`, `len()`, `split()`.

Opérateurs

- Comparer des valeurs avec `==`, `!=`, `<`, `>`, `<=`, `>=`.
- Combiner ou inverser des conditions avec `and`, `or`, `not`.

Structures de contrôle

- Conditions (`if`, `elif`, `else`) : exécuter du code selon qu'une condition est vraie ou fausse.
- Boucles (`for`, `while`) : répéter des instructions en parcourant une séquence ou tant qu'une condition est vraie.



Pour aller plus loin...

Pratiquer

Exercez-vous à écrire des très programmes simples qui utilisent les concepts appris.

Entraînez vous

<https://www.w3schools.com/python/>

-> 'Python For Loops'

The screenshot shows the W3Schools Python Tutorial page. The top navigation bar includes links for Tutorials, References, Exercises, and Certificates, along with a search bar and various utility links like Spaces, For Teachers, Upgrade, Get Certified, and Sign In. The main content area is titled 'Python Tutorial' and features a green 'Learn Python' section with a 'Start learning Python now »' button. A sidebar on the left lists various Python topics, with 'Python HOME' highlighted. A right sidebar contains a 'Python Learning Resources' section with a 'CHECK IT OUT!' button, a 'COLOR PICKER' section with a color wheel, and social media links. A tip at the bottom suggests signing in to track progress.

Python Tutorial

Python HOME

Python Intro

Python Get Started

Python Syntax

Python Comments

Python Variables

Python Data Types

Python Numbers

Python Casting

Python Strings

Python Booleans

Python Operators

Python Lists

Python Tuples

Python Sets

Python Dictionaries

Python If...Else

Python Match

Python While Loops

Python For Loops

Python Functions

Python Decorators

Python Range

Python Tutorial

Learn Python

Python is a popular programming language.

Python can be used on a server to create web applications.

Start learning Python now »

Tip: [Sign in](#) to track your progress - it's free.

Learning by Examples

With our "Try it Yourself" editor, you can edit Python code and view the result.

Save the date!



**ARCHIVES
NATIONALES**

X



Maison
des Sciences
de l'Homme
**MSH
MONDES**

16-18 DÉCEMBRE 2026
PIERREFITTE-SUR-SEINE - SAINT-DENIS

Métro **13** station Saint-Denis Université
59, rue Guynemer
Pierrefitte-sur-Seine
93383 Saint-Denis



HACKATHON

ARCHIVES NATIONALES × PICTORIA

EXPLORATION MULTIMODALE DES ARCHIVES
ICONOGRAPHIQUES ET AUDIOVISUELLES

NOUVEAUX REGARDS



ARCHIVES

+



INTELLIGENCE
ARTIFICIELLE

+



CO-CONSTRUCTION



Scannez pour
en savoir plus

